

OS_2022-04-08

1) Synchronisation mit Semaphoren (30)

In einer Lackierkammer werden Bahnwaggons mit Roboterunterstützung lackiert. Zwei Prozesse kontrollieren den Lackiervorgang, bei dem Waggons einzeln lackiert werden.

- Der Prozess *main_control* bringt Waggons in die Lackierkammer (*load_waggon()*) und transportiert sie nach dem Lackieren aus der Kammer (*remove_waggon()*).
- Der Prozess *paint_robot* führt die einzelnen Schritte des Heißlackierens durch einen Lackierroboter durch: der Roboter füllt zunächst seinen Farbtank an der Befüllungsanlage (*fill_paint()*), heizt dann den Farbwärmer auf (*warm_up()*), lackiert den Waggon (*paint()*) und kühlt am Ende den Farbwärmer ab (*cool_down()*).

Die Aktionen der Prozesse und die Synchronisation zwischen den Prozessen sehen Sie in den folgenden Programmstücken.

```
void main_control(void)
{
    while(test_work_to_do())
    {
        load_waggon();
        V(waggon_ready);
        /* waggon painting */
        P(waggon_painted);
        remove_waggon();
    }
}

void paint_robot(void)
{
    P(waggon_ready);

    fill_paint();
    warm_up();
    paint();
    cool_down();

    V(waggon_painted);
}
```

Um das Lackieren zu beschleunigen, wird der Roboterprozess durch drei parallele Roboterprozesse ersetzt.

Fügen Sie in den Code-Skeletten für die Roboterprozesse *Semaphoroperationen* ein, um die Roboterprozesse mit dem Prozess *main_control* und untereinander zu synchronisieren. Geben Sie geeignete Initialisierungen für alle Semaphore an. Ihre Synchronisationskonstrukte sollen folgende Punkte sicherstellen:

- Der Code von *main_control* bleibt unverändert.
- Die Roboter beginnen erst dann mit ihren Aktionen, wenn ein Waggon in der Lackierkammer ist. Der Abtransport des Waggons darf erst beginnen, wenn alle drei Roboter mit all ihren Arbeitsschritten fertig sind.
- An der Farbbefüllungsanlage kann zu jedem Zeitpunkt maximal ein Roboter aufgetankt werden. Die Befüllungsreihenfolge der Roboter soll nicht künstlich durch die Software eingeschränkt werden.
- Die Roboter beginnen gemeinsam mit dem Lackieren, wenn alle Roboter mit dem Aufheizen der Farbvorwärmer fertig sind. Das Lackieren durch die Roboter soll dann parallel erfolgen.

Initialisierungen:

```
init(waggon_ready, 0);

int count_warm = 0
int count_done = 0
```

```
init(mutex_fill, 1);
init(mutex_count= 1);
init(waggon_painted, 0);
```

```
init(startrob, 0);

init(paint_go, 0);
init(done, 0);
```

void paint_robot1(void)

{

```
wait(waggon_ready);
signal(startrob);
signal(startrob);

wait(mutex_fill);
```

fill_paint();

signal(mutex_fill)

warm_up();

```
wait(mutex_count);
count_warm++;
if (count_warm == 3) {
    signal(mutex_count);
    count_warm = 0;
    signal(paint_go);
    signal(paint_go);
} else {
    signal(mutex_count);
    wait(paint_go);}
```

paint();

cool_down();

```
wait(mutex_count);
count_done ++;
if (count_done == 3) {
    signal(mutex_count);
    count_done = 0;
    signal(done);
    signal(done);
    signal(waggon_painted)
} else {
    signal(mutex_count);
    wait(done);
}
```

void paint_robot2(void)

{

```
wait(startrob);

wait(mutex_fill);
```

fill_paint();

signal(mutex_fill)

warm_up();

```
wait(mutex_count);
count_warm++;
if (count_warm == 3) {
    signal(mutex_count);
    count_warm = 0;
    signal(paint_go);
    signal(paint_go);
} else {
    signal(mutex_count);
    wait(paint_go);}
```

paint();

cool_down();

```
wait(mutex_count);
count_done ++;
if (count_done == 3) {
    signal(mutex_count);
    count_done = 0;
    signal(done);
    signal(done);
    signal(waggon_painted)
} else {
    signal(mutex_count);
    wait(done);
}
```

void paint_robot3(void)

{

```
wait(startrob);

wait(mutex_fill);
```

fill_paint();

signal(mutex_fill)

warm_up();

```
wait(mutex_count);
count_warm++;
if (count_warm == 3) {
    signal(mutex_count);
    count_warm = 0;
    signal(paint_go);
    signal(paint_go);
} else {
    signal(mutex_count);
    wait(paint_go);}
```

paint();

cool_down();

```
wait(mutex_count);
count_done ++;
if (count_done == 3) {
    signal(mutex_count);
    count_done = 0;
    signal(done);
    signal(done);
    signal(waggon_painted)
} else {
    signal(mutex_count);
    wait(done);
}
```

andere Lösung

Initialisierungen:

init(waggon_ready, 0);

init(paint_done1, 0);

init(waggon_painted, 0);

```
init(startrob, 0);
init(mutex_fill, 1);
```

```
init(paint_done2, 0);
init(paint_done3, 0);
```

```
init(waggon_painted, 0);
```

```
void paint_robot1(void)
```

```
{
```

```
wait(waggon_ready);
signal(startrob);
signal(startrob);

wait(mutex_fill);
```

```
fill_paint();
```

```
signal(mutex_fill);
```

```
warm_up();
```

```
signal(paint_done1);
signal(paint_done1);
```

```
wait(paint_done2);
wait(paint_done3);
```

```
paint();
```

```
cool_down();
```

```
wait(paint_done2);
wait(paint_done3);

signal(waggon_painted);
```

```
}
```

```
void paint_robot2(void)
```

```
{
```

```
wait(startrob);

wait(mutex_fill);
```

```
fill_paint();
```

```
signal(mutex_fill);
```

```
warm_up();
```

```
signal(paint_done2);
signal(paint_done2);
```

```
wait(paint_done1);
wait(paint_done3);
```

```
paint();
```

```
cool_down();
```

```
signal(paint_done2);
```

```
}
```

```
void paint_robot3(void)
```

```
{
```

```
wait(startrob);

wait(mutex_fill);
```

```
fill_paint();
```

```
signal(mutex_fill);
```

```
warm_up();
```

```
signal(paint_done3);
signal(paint_done3);
```

```
wait(paint_done1);
wait(paint_done2);
```

```
paint();
```

```
cool_down();
```

```
signal(paint_done3);
```

```
}
```

2) Deadlock Avoidance - Banker's Algorithm (25)

In einem Computersystem gibt es drei Prozesse (P_1, P_2, P_3) und drei Arten von Ressourcen (R_1, R_2, R_3). Der *Resource Vector* $R = (7, 8, 7)$ beschreibt die Anzahl der vorhandenen Ressourcen. Prozesse verwenden die Operationen $get(r_1, r_2, r_3)$ bzw. $free(r_1, r_2, r_3)$, um r_i Ressourcen der Ressourcenart R_i ($i = 1 \dots 3$) anzufordern bzw. freizugeben.

Die folgende Abbildung zeigt für jeden der drei Prozesse die Folge von get und $free$ -Operationen, die bei jeder Prozessabarbeitung durchgeführt werden.

P_1	$get(2, 1, 1)$ $free(2, 1, 0)$ $get(1, 2, 2)$ $get(2, 1, 0)$ $free(2, 1, 1)$ $get(1, 0, 3)$ $free(1, 2, 2)$ $free(1, 0, 3)$	P_2	$get(1, 1, 1)$ $get(1, 0, 0)$ $free(1, 1, 1)$ $get(0, 3, 0)$ $get(1, 0, 0)$ $free(1, 0, 0)$ $get(1, 0, 0)$ $free(2, 3, 0)$	P_3	$get(0, 2, 2)$ $free(0, 1, 0)$ $get(1, 1, 1)$ $free(0, 0, 1)$ $get(0, 0, 1)$ $get(3, 1, 0)$ $free(2, 2, 2)$ $free(2, 1, 1)$
-------	--	-------	---	-------	--

Nehmen Sie an, dass die drei ersten Operationen jedes Prozesses abgearbeitet wurden (d.h., die Abarbeitung jedes Prozesses befindet sich an der strichlierten Linie). Als nächster Schritt soll die vierte Operation von P_1 (grau hinterlegte Operation) durchgeführt werden.

Verwenden Sie den Banker's Algorithmus, um festzustellen, ob die vierte Operation von P_1 durchgeführt werden soll. Geben Sie alle erforderlichen Vektoren und Matrizen an und führen Sie den Banker's Algorithmus schrittweise durch, d.h. geben Sie für jeden Schritt die Werte der Elemente aller relevanten Matrizen und Vektoren an.

$$R = (7, 8, 7)$$

$$A_1 = (1, 2, 3) \quad A_2 = (1, 0, 0) \quad A_3 = (1, 2, 3)$$

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 0 & 0 \\ 1 & 2 & 3 \end{pmatrix}$$

$$V = R - A_1 - A_2 - A_3$$

$$V = (4, 4, 1)$$

$$C_1 = (3, 3, 5)$$

$$C_2 = (2, 3, 1)$$

$$C_3 = (4, 3, 3)$$

$$C = \begin{pmatrix} 3 & 3 & 5 \\ 2 & 3 & 1 \\ 4 & 3 & 3 \end{pmatrix}$$

$$N = C - A = \begin{pmatrix} 2 & 1 & 2 \\ 1 & 3 & 1 \\ 3 & 1 & 0 \end{pmatrix}$$

$$Q_1 = (2, 1, 0)$$

$$\begin{array}{l|l} Q_1 \leq N_1? & \checkmark \\ Q_1 \leq V? & \checkmark \end{array}$$

P_1

$$A'_1 = A_1 + Q_1 = (3, 3, 3)$$

$$N'_1 = N_1 - Q_1 = (0, 0, 2)$$

$$W = V - Q_1 = (2, 3, 1)$$

Prüfung P_2

$$N_2 = (1, 3, 1)$$

$$N_2 \leq W? \checkmark$$

P_2 beenden

$$\text{neues } W = W + A_2 = (3, 3, 1)$$

Prüfung P_3

$$N_3 = (3, 1, 0) \leq W? \checkmark$$

$P_3 \rightarrow$ beendet

$$W = W + A_3 = (4, 5, 4)$$

$P_{1,2}$:

$$N_1 = (0, 0, 2)$$

$$N_1 \leq w \quad \checkmark$$

$\rightarrow P_1$ been

$$w = 11^1 + w = \underline{\underline{(\neg 87)}}$$

Reihenfolge (P_2, P_3, P_1)

3) Fragen zu Betriebssystemen

Was versteht man unter einem *Process Image*? Erklären Sie, aus welchen Teilen ein Process Image besteht? (4)

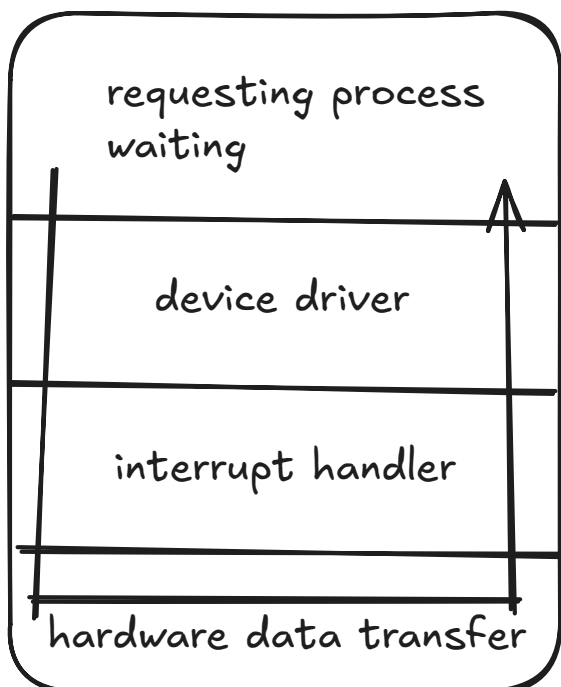
- Das OS schreibt den derzeitigen Status/Aufbau eines Prozesses, braucht das OS für die Speicherverwaltung von RAM
- Die Process Tables vom OS zeigen auf Process Images
- Liegen im Virtual Memory

Besteht aus :

- Process Control Block (Process identification, processor state information, process control information)
- User Programm
- User Data (User Stacks)
- System Stacks

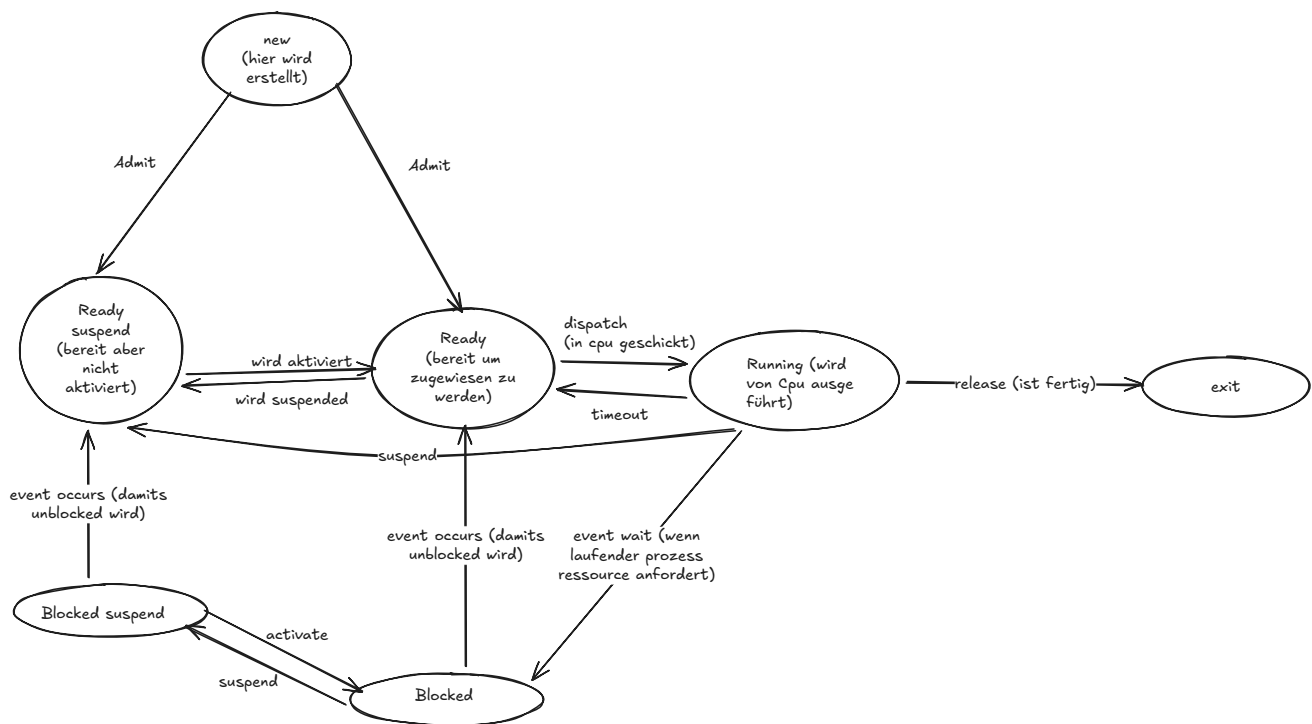
Skizzieren Sie die Folge der Schritte, die bei der Abarbeitung eines *Synchronen I/O Requests* ablaufen. (5)

[:bs07_input-output,p.13](#)



Zeichnen Sie das Zustandsdiagramm eines Prozesses, beginnend mit der Entstehung, bis zur Beendigung des Prozesses. Geben Sie die Namen der Zustände an und beschreiben Sie kurz jeden Zustand und Zustandsübergang. (5)

[:theorie_fragen_ausarbeitung,p.1|600](#)



Nennen Sie Möglichkeiten, um in einem Paging-System Speicherschutz zu realisieren? (3)

- Bound Check: Logischer Adressbereich ist kontinuierlich zusammenhängend, man überprüft ob Adresse innerhalb der maximalen Länge des Adressbereichs liegt.
- Bei Speicherbereichs-Sharing: Protection Keys
 - Variante 1: Jeder (physische) Frame hat einen Key, jeder Prozess hat einen Key, bei Adressierung auf Frame wird Key abgeglichen
 - Variante 2: Jeder Prozess hat Menge von Keys, in TLB ist zu jeder logischen Adresse ein Key hinterlegt, wenn einer der Prozess keys mit TLB Key übereinstimmt -> Zutritt
 - Sonst immer: Speicherbereichsverletzung
- Acc (Table mit Zugriffsrechten -> "wer darf was?")

Erklären Sie die Begriffe *absoluter Pfadname* und *relativer Pfadname* und geben Sie jeweils ein Beispiel an. (2)

Absoluter Pfadname beginnt bei der root und relativer Pfadname beginnt in der derzeitigen directory:

- Absoluter Pfad: /etc/folder1/file.txt
- Relativer Pfad: folder1/file.txt

Welcher Vorteil ergibt sich aus der Einführung von *Threads* für den Benutzer? Wodurch kommt es zu diesem Vorteil? Worauf muss der Benutzer bei der Programmierung von Threads achten? (3)

Geschwindigkeit, da Teile von Prozessen parallel laufen können. Das funktioniert, da Threads einen gemeinsamen Speicher haben. Man muss deshalb auch bei der Programmierung darauf achten, dass man nicht eine Änderung im Speicher direkt wieder mit nicht aktuellen Werten überschreibt, da man ja gemeinsamen Speicher hat -> Race condition.

Bei der Realisierung von Dateisystemen gibt es verschiedene Möglichkeiten, um die zu einer Datei gehörenden Datenblöcke zu organisieren bzw. auffindbar zu machen (Block-Allokierung). Nennen Sie vier verschiedene Strategien zur Block-Allokierung von Dateien und beschreiben Sie diese mit ihren Vor- und Nachteilen. (5)

- Contiguous Allocation
 - Vorteil: Gute Performance beim Lesen weil Festplatten Kopf sich kaum bewegen muss
 - Nachteil: Schwierig Dateien nachträglich zu vergrößern
- Chained Allocation:
 - Datei belegt einzelne Blöcke die nicht zusammenhängen müssen aber wie bei linked list wird zum nächsten gezeigt
 - Vorteil: keine externe Fragmentierung, Dateien lassen sich leicht erweitern
 - Nachteil: Seektime höher weil Dateien verstreut
- indexed Allocation:
 - Vorteile: Direkte als auch esequentielle Zugriff gut unterstützt
 - Nachteil: index-Tabelle (Fat) verbraucht viel Platz im RAM
- i-nodes:
 - Vorteil: i-node muss nur im RAM sein wenn Datei tatsächlich verwendet wird
 - Nachteil: Anzahl der Blockreferenzen in einem I-Node ist begrenzt, Große Dateien müssen über mehrere Blockschichten gehen...

Nennen Sie vier Designprinzipien für die Konstruktion von sicheren Systemen. Geben Sie für jedes Prinzip ein Beispiel an. (4)

1. Open-Design: keine Sicherheit durch besonders schwere Codes -> muss verständlich bleiben
2. Default Einstellungen: keine Berechtigungen -> BSP User soll default mäßig kein Admin haben
3. Least Privilege: so wenig Rechte wie möglich -> User soll nur das machen können, was er wirklich machen muss, alles andere sollte nicht funktionieren
4. Überprüfung der Berechtigungen -> Auch ein halbes Jahr später sollte geprüft werden ob jeder die selben Rechte hat die anfangs zugewiesen worden sind.

Beschreiben Sie die folgenden Strategien für das CPU Scheduling und vergleichen Sie deren Eigenschaften: *FCFS*, *Round Robin*, *SPN* und *SRT*. (4)

FCFS

- Einfach eine Queue jeder der kommt kommt in der Reihenfolge drann in der er in die Warteschlange gegangen ist
- benachteiligt kürzere Prozesse wenn ein großer Prozess vor ihnen in die Queue gekommen ist.

Round Robin

- Es wird immer zwischen den offenen Prozessen hin und her gewitched und immer möglichst gleich lang ein Prozess bearbeitet bis zum nächsten gewitched wird

- Alle werden gleichmäßig behandelt
- Benachteiligung von IO intensiven Programmen die viel Zeit bei IO Operationen verschwenden

SPN

- Immer der kürzeste Prozess wird als nächstes genommen.
- Starvation von langen Prozessen möglich.

SRT

- Shortest remaining time
- Wenn ein kürzerer Prozess kommt als der aktive, wird gewitched
- Gute Behandlung von kurzen Prozessen
- Starvation von langen Prozessen möglich

Beschreiben Sie Aufgabe und Funktion eines *Translation Lookaside Buffers*? Worauf hat man bei der Betriebssystemimplementierung bei einem Process Switch zu achten, wenn man einen Translation Lookaside Buffer verwendet? (3)

- Aufgabe: Spezieller Hardware Cache innerhalb der MMU (memory management unit) um die Latenz der Adressübersetzung zu minimieren. Ohne TLB müsste bei jedem Speicherzugriff erst die Seitentabelle im RAM gelesen werden...
- Funktion: möchte man die Adressübersetzung von virtuell zur physischen Adresse, schaut man zuerst mit dem ersten Teil der virtuellen Adresse (#Page) in der TLB nach und sucht nach der passenden Page number. Findet man eine (TBL Hit), so hat man die assoziative Frame number und muss nicht mehr in den Page table schauen. Offset bleibt gleich
- Bei Process Switch acht geben auf: Wenn man einen Process switch hat, dann muss die TLB gelöscht werden, das heißt man bekommt am Anfang eher seltener einen TBL hit.

Mit welcher logischen Struktur des I/O Systems versucht man bei der Realisierung von I/O Funktionen sowohl ein einheitliches Programmierinterface, als auch eine möglichst gerätespezifische Ansteuerung zu erreichen? (4)

Mit der Schichtstruktur, aufgeteilt in:

- Logische I/O
- Geräte I/O
- und Scheduling und Control

Beschreiben Sie das typische Layout einer Disk bzw. eines Filesystems. Erklären Sie kurz die einzelnen Komponenten. (3)

- Master boot record: kümmert sich beim booten darum, dass die aktive Partition ihren boot block die Kontrolle gibt
- partition table: hat die meta daten der Partitionen
- Partitionen:
 - boot block (führt code aus der dann den Kernel des OS lädt)

- super block hat Metadaten der partition
- free management: verwaltet Belegung der Daten
- i-nodes: Datenstruktur für die Metadaten einer einzelnen Datei
- root-dir: Einstiegspunkt für File tree
- files und directories